Compiler-Enforced Typestate Boundaries for Hostile-Silicon and Physical Fault-Injection Threat Models

A Specification and Structural-Proof Analysis of the Ghost-Hunter v1.0.18 Architecture

Taha Kouiyasse
Systems Architect
tahakouiyasse@protonmail.com

Document version 1.0 — July 2026

Abstract

This document specifies Ghost-Hunter v1.0.18, a declarative architecture for an inline cryptographic boundary controller targeting the Hubris microkernel, and reports the results of three independently executable proofs of concept (PoCs) verifying three of its structural security claims. The threat model under consideration excludes the operating system and application layers as trust boundaries and instead treats the host CPU platform, its management subsystems, and the physical memory bus as potentially adversarial. This includes: a compromised or vendor-backdoored CPU management engine with independent network egress; a Ring -3 or DMA-capable adversary with read access to physical RAM; and an adversary with the capability to induce single-event-upset-class faults (voltage glitching, laser fault injection) against the target silicon during a security-critical read. Standard trusted-execution technologies (Intel SGX, AMD SEV) are examined and shown to assume trustworthy silicon as an axiom, which this threat model explicitly does not grant. Ghost-Hunter's response is architectural rather than cryptographic: a set of Rust type-system and linker-level constraints that convert specific classes of implementer error and hardware fault into either compile-time rejections or exhaustively-typed runtime error states, rather than relying on programmer discipline or best-effort defensive coding. **Technology readiness disclosure, stated here rather than deferred:** as of this writing, Ghost-Hunter v1.0.18 is a declarative specification with zero implemented function bodies and zero executable runtime logic — this status is stated explicitly in the specification document's own header. What exists and has been independently verified are three standalone PoCs, decoupled from the Hubris target and runnable on commodity Linux hardware, each proving one narrow, named structural property: (1) compiler rejection of three independent attempts to exfiltrate a reconstructed key past its authorized scope; (2) ELF-binary-level confirmation, via `nm`, `objdump`, and `readelf` against a compiled and linked artifact, that two secret-key shares occupy non-overlapping, page-aligned memory regions; and (3) a 200,000-trial single-bit fault-injection simulation showing zero silent security-check bypasses against a Hamming-distance-32 encoded boolean, contrasted against a 100% bypass rate for a naive one-bit-apart encoding under an identical fault model. Each claim is reported with its explicit boundary: what is proven, and what remains outside the proof's reach.

1 The Failure of Current Technologies

The premise of this document is narrow and should not be read more broadly than stated: existing hardware trusted-execution technologies and existing operating-system-level isolation mechanisms are not designed to hold under a threat model in which the CPU platform itself, or a subsystem with privilege above the OS kernel, is assumed adversarial. This is not a defect in those technologies — it is outside their stated design goal. The failure being described here is a failure of applicability to a specific threat model, not a failure of engineering.

1.1 Hardware Trusted Execution Assumes Trustworthy Silicon

Intel SGX and AMD SEV both construct their security guarantees on top of a foundational, unstated axiom: that the CPU package executing the enclave or the encrypted-memory logic is itself faithfully implementing its documented specification, free of an undocumented debug interface, a vendor-inserted backdoor, or a supply-chain-compromised die. Both technologies protect a workload from a hostile *operating system* or hostile *hypervisor* running alongside it. Neither technology's threat model extends to protecting a workload from the *CPU package that is executing the enclave's own instructions.* If an adversary controls the silicon — through an undisclosed hardware debug feature, a firmware-level backdoor in a coupled management subsystem, or a fabrication-stage implant — SGX's memory-encryption engine and SEV's guest-encryption keys are administered by that same silicon, and the guarantee collapses at its root. This is a structural property of the trust model, not an implementation bug correctable by a future SGX or SEV revision: any protection mechanism that is administered by the component you are trying to distrust cannot, by construction, defend against that component.

1.2 Standard Operating Systems Do Not Defend Against Ring −3

The privilege-ring model taught in most operating systems curricula (Ring 0 for kernel, Ring 3 for userspace) does not include Intel's Management Engine (ME) or AMD's Platform Security Processor (PSP), both of which execute with privilege above Ring 0, independent of and invisible to the host operating system, on separate embedded cores with their own firmware, their own memory access paths, and in ME's case, historically, independent network stack access coupled to specific integrated network interface silicon. A standard Linux kernel — however hardened, however aggressively patched — has no visibility into and no control over this subsystem's behavior, because it does not run in a ring the kernel can observe or a privilege domain the kernel can enforce a boundary against. Kernel-level mitigations (SELinux policy, seccomp-bpf, kernel lockdown mode) constrain what Ring 0 and Ring 3 code can do to each other. None of them constrain a subsystem that was never inside that boundary to begin with. A Ring −3-capable adversary with DMA read access to physical memory does not need to defeat the kernel's protections, because those protections were never positioned to apply to it.

1.3 Standard Cryptographic Practice Assumes an Honest Compiler and Honest Silicon

Two failure classes specific to applied cryptography, as distinct from cryptographic theory, are relevant here and are frequently underweighted in practice.

Dead-Store Elimination (DSE). A cryptographic key held in a stack buffer, once no longer read, is — from the perspective of an optimizing compiler's data-flow analysis — a dead value. A naive zeroization routine (a plain loop writing zero bytes to the buffer) writes to memory that nothing subsequently reads before the buffer's storage is deallocated or reused. Under the "as-if" rule that governs optimizing compilers, a write with no observable effect on the program's output is a candidate for elimination, and LLVM-based toolchains routinely eliminate exactly this pattern at moderate to aggressive optimization levels. The consequence is a zeroization call that is present, compiles, appears in the source, and does nothing in the shipped binary — silently, with no compiler warning, because the elimination is correct according to the language's own execution model. Cryptographic key material believed destroyed remains resident in memory for an attacker with a subsequent read primitive (a use-after-free, a stack-scanning technique, a cold-boot attack) to recover.

Physical fault injection. Voltage glitching and laser fault injection are established techniques, documented extensively in the hardware security literature, for inducing a controlled bit-level fault in a target device's memory or register state at a chosen instant, typically timed against a security-critical comparison or branch. A verification routine that encodes its pass/fail

result as an ordinary `bool` or a one-bit-apart integer pair (commonly `0` for false, `1` for true) presents a single-bit target: one successfully induced fault, timed correctly, silently converts a failed verification into a passed one, with no error state, no exception, and no observable anomaly in the program's control flow. Standard cryptographic libraries, correctly implementing the mathematics of the algorithms they provide, are not typically designed to consider this failure mode, because it is a hardware-adjacent concern, not an algorithmic one — and the encoding choice that creates the vulnerability (an ordinary boolean) is usually made without the fault-injection threat model in view at all.

```
Scope boundary, stated once here and applicable throughout this document:  none
of the above is a claim that SGX, SEV, Linux, or standard cryptographic libraries
are defectively engineered.  It is a claim that a specific, narrow, and admittedly
extreme threat model -- hostile silicon, a Ring −3-capable physical adversary, and
an adversary capable of physical fault injection -- falls outside all of their design
envelopes simultaneously, and that no combination of them, deployed together, closes
that gap.
```

2 The Ghost-Hunter Protocol: Architecture Under an Assume-Hostile-Silicon Model

Ghost-Hunter v1.0.18 is a declarative specification for an inline cryptographic boundary controller, targeting Oxide Computer Company's Hubris microkernel, structured as a set of Rust task crates with the properties described below. The specification's own header states its status without qualification: *"Declarative specification only. Zero runtime logic. Zero function bodies."* This section describes the architecture as specified. Section 6 addresses, with equal directness, what has and has not been independently verified.

2.1 Microkernel Isolation and Network-Egress Starvation

The target platform is Hubris, a microkernel architecture in which each task is a statically-declared, memory-isolated unit with no dynamic memory allocation, no filesystem, no POSIX file descriptors, and no facility for one task to remap another's virtual memory. This is a stronger isolation baseline than a general-purpose OS process model provides by default, because the isolation guarantees are enforced by the kernel's static task table rather than by runtime access-control checks that a sufficiently privileged adversary (Ring −3 or otherwise) might bypass.

Against the ME/AMT network-egress concern described in Section 1.2, the specification's mitigation is narrow and should be described exactly as narrow: the network-facing task (`task-gh-filter`) is bound to a discrete, PCIe-attached network interface (`pcie_ext_nic` in the specification's driver naming) rather than an integrated NIC historically coupled to ME's out-of-band management network path. This closes one specific, documented network-egress route associated with specific integrated silicon. It is not, and the specification does not claim it to be, a general neutralization of the Management Engine as a subsystem, nor a claim about undocumented capabilities of Management Engine silicon beyond the specific network-coupling behavior being routed around.

2.2 Anti-DMA Page Splitting

The specification's key material is not stored as a single contiguous secret. Instead, an HMAC key is split via a two-share XOR scheme (`KeyShareA`, `KeyShareB`) into two independently-declared Rust statics, each carrying the attribute `#[repr(C, align(4096))]`, placing each share on its own 4096-byte-aligned memory page. The stated goal is narrowing — the specification's own invariant table uses the word "narrowing (not eliminating)" — the window in which the complete key is present at a single, DMA-readable memory location. A physical-memory read that captures

a single naturally-aligned 4096-byte page captures, at most, one share; the complete key requires two separate reads at two separate addresses.

This claim is the subject of PoC 2 (Section 6.2), and the PoC's own findings materially refine the claim beyond what the source attribute alone would suggest: the `#[link_section]` attribute used to name each share's storage is honored by the compiler at the object-file level but is not guaranteed to survive as a distinct *output* section after final linking, because standard linker behavior merges same-prefixed `.bss.*` input sections into a single output `.bss`. The property that survives to the shipped artifact and that actually enforces page separation is the `align(4096)` constraint on the type itself, independently of section-name survival. Section 6.2 reports this distinction as verified directly against a compiled and linked ELF binary.

2.3 The Closure Enclosure: Borrow-Checker-Enforced Key Destruction

Section 5.6 of the specification (new in v1.0.18) replaces a prior pattern in which calling code could obtain direct references to both key shares and perform XOR reconstruction itself — a pattern that places no compiler-enforced limit on how long the reconstructed key value can be retained, copied, or passed onward. The v1.0.18 replacement is a combinator, `with_reconstructed_key`, which:

- Reconstructs the key internally and constructs an opaque `SecretKeyBuffer` value.
- Passes a reference to that value into a caller-supplied closure, `FnOnce(&SecretKeyBuffer) -> R`.
- Drops (and, via a `Drop` implementation, volatile-zeroizes) the `SecretKeyBuffer` unconditionally at the end of the combinator's own stack frame, after the closure returns an owned value `R` with no lifetime tying it back to the buffer's scope.

The `SecretKeyBuffer` type implements neither `Copy` nor `Clone`, exposes no field access, and implements no generic conversion trait (`AsRef<[u8]>`, `Deref<Target = [u8]>`, `Borrow<[u8]>`) that would allow the raw bytes to be reached by any path other than a function explicitly typed to accept `&SecretKeyBuffer`. `KeyShareA` and `KeyShareB` additionally have no `expose()` method in this revision — the method existed and returned raw share bytes in the prior specification revision and has been deleted, not merely deprecated or hidden.

The specification is explicit, and this document preserves that explicitness rather than rounding it up, about the boundary of what this closes: during the closure's execution, the reconstructed key exists as an ordinary, contiguous 32-byte value in real stack memory, indistinguishable at the physical-memory level from the prior, unsafe pattern's exposed key. A DMA-capable or Ring −3 adversary with read access during that specific execution window observes the same bytes regardless of which pattern produced them. What this mechanism closes is a *software* escape — the class of implementer error in which a well-intentioned caller retains, copies, or forwards a reference to key material beyond its intended scope. It does not, and cannot, shrink the physical exposure window during legitimate use. This distinction is verified in PoC 1 (Section 6.1).

2.4 Fault-Injection Immunity: Hamming-Distance-32 Encoding

The specification's `GlitchResistantBool` type replaces an ordinary boolean encoding for a security-critical verification result with a `#[repr(u32)]` enum whose two variants are defined as exact 32-bit bitwise complements:

```
#[repr(u32)]
pub enum GlitchResistantBool {
    True  = 0x5A5A5A5A,
```

```
    False = 0xA5A5A5A5,
}
```

Decoding from a raw `u32` (as received, for instance, over a Hubris IPC channel, which copies bytes without validating that they represent one of the enum's defined discriminants) is mandatory and fallible: the type does not implement `From<u32>` — an infallible conversion is explicitly banned by the v1.0.18 specification text — and instead implements `TryFrom<u32>`, returning `Result<GlitchResistantBool, GlitchDecodeError>`, where `GlitchDecodeError` is a unit-like enum with exactly one variant, `Undefined`, produced for any input that is neither of the two defined sentinel values.

Because the two sentinels differ in every one of their 32 bits (Hamming distance 32, the maximum possible for a 32-bit value), a single-bit fault applied to either sentinel cannot, by the arithmetic of the encoding, produce the other sentinel: reaching the other defined value from a single starting sentinel requires flipping all 32 bits simultaneously. A single-bit fault instead produces one of the $2^{32}-2$ undefined bit patterns, which the mandatory `TryFrom` implementation classifies as `Err(GlitchDecodeError::Undefined)` rather than silently defaulting to either `True` or `False`.

The specification states, and this document again preserves rather than omits, the precise boundary of this guarantee: it protects the *decode instant* only. A fault landing on an already-decoded `GlitchResistantBool` value sitting in a register or stack slot between a successful decode and the value's next read is not addressed by this type and cannot be addressed by any software-level type definition — that failure mode is a hardware fault-coverage question (ECC, redundant read-back, watchdog timers), explicitly out of scope for a `no_std` task crate. This property, and its boundary, are verified empirically in PoC 3 (Section 6.3).

2.5 The Hardware Honeypot: `BAIT_KEY`

The specification defines a single private static, `BAIT_KEY`, an easily recognizable 32-byte sentinel pattern (`0xDEADBEEF` repeated) placed in its own named link section (`.bss.cleartext_bait`). The specification is unambiguous that this value **must never be read, used, or referenced by any verification logic in the workspace** — it is not an active check the code performs, but a passive tripwire whose entire security value derives from an external observer's tooling (a memory-scanning or RAM-dumping technique associated with the platform-level threat this mechanism targets) encountering it during a scan of process or physical memory. Because no legitimate code path in the specification ever touches `BAIT_KEY`, its appearance in any capture of runtime memory state is, by construction, evidence of external scanning activity rather than a false positive against normal operation.

The specification places an equally explicit negative constraint alongside this mechanism: `BAIT_KEY` must not become, under any circumstance or future revision, a detector for or response to legitimate operator debugging (Humility probe attachment, task dump, IPC trace) — the specification's design invariants table states plainly that no task in the workspace may implement anti-debug, anti-inspection, or monitoring-suppression logic under any name. `BAIT_KEY`'s isolation from the rest of the verification logic is intended to be structural (a private static, unreachable from application code, verified via a compile-fail stub in the specification's own test suite) rather than incidentally achieved, precisely so that it cannot later be repurposed, deliberately or through code drift, into exactly the anti-inspection mechanism the specification forbids.

3 Target Applications and Buyer Categories

This section describes the threat model Ghost-Hunter is architected to eventually address, and the categories of organization for whom that threat model constitutes an actual, unmet opera-

tional requirement — not a claim of current deployment, certification, or flight heritage, none of which apply to a zero-runtime specification. The distinction matters and is maintained deliberately throughout this section.

Military-grade Hardware Security Modules. HSM procurement in defense contexts frequently specifies threat models — supply-chain-compromised silicon, adversarial physical access during transport or storage, laser fault-injection resistance during red-team evaluation — that commercial HSM certification regimes (FIPS 140-2/3, Common Criteria) do not fully address, because those regimes generally assume the certified module's own silicon is trustworthy by definition of having passed certification. An architecture whose security claims are structural (compiler- and linker-enforced) rather than trust-assumption-based is a closer match to this requirement category than an architecture relying on the correctness of a black-box secure element.

Space-grade satellite uplink cryptography. Radiation-induced single-event upsets are a documented, unavoidable operating condition for space-grade electronics, not merely an artificially induced attack scenario — which makes a fault-injection-immune verification encoding directly relevant to a real, continuously-present operating environment rather than a hypothetical adversary. This category is noted here as an architectural fit for the *fault-injection-immunity* component specifically; no claim is made regarding the platform's readiness for space-grade qualification, which requires additional certification, radiation testing, and hardware-specific validation entirely outside this specification's current scope.

Zero-trust critical infrastructure. Operational technology environments (energy grid control systems, industrial control systems) increasingly specify zero-trust requirements that extend, in principle, to distrust of the underlying compute platform's own management subsystems — the same threat category Section 1.2 describes. An architecture that assumes hostile silicon by design, rather than treating platform trust as an unexamined baseline, addresses a documented gap in current zero-trust reference architectures, most of which stop at the OS/hypervisor boundary.

Intelligence-agency secure gateways. Gateway systems mediating access between classification domains are a natural fit for a threat model that assumes both physical adversarial access and platform-level compromise, since these are exactly the conditions such gateways are typically designed against in the first instance.

```
This section names use cases where the threat model is relevant.  It does not
claim procurement readiness, certification, or existing deployment in any of these
categories.  An organization operating in any of the above categories evaluating
this architecture should expect to perform the same due diligence it would apply
to any pre-implementation specification, and should treat Section 6 as the accurate
description of what currently exists.
```

4 Structural Proofs Over Runtime Promises

The specification's own header states its status without hedge: *declarative specification only, zero runtime logic, zero function bodies.* No claim in this document should be read as contradicting that. What follows is a complete account of what does exist and has been independently, reproducibly verified: three standalone proofs of concept, decoupled entirely from the Hubris target and the reference N4500 hardware, each written as an ordinary Rust project runnable on any Linux machine with a stable toolchain. Each PoC targets exactly one structural claim from the specification, proves it against a real compiler or linker rather than against prose, and reports its own boundary — what the result does and does not establish — rather than allowing a clean result to imply more than it earned.

The distinction this section is built around, stated directly: a specification describes intent. A PoC that fails to compile when it should, or that reports a non-zero bypass count when the claim requires zero, is not describing intent — it is a fact about what a real compiler, a real

linker, or a real 200,000-iteration simulation actually did, independent of anyone's belief about the underlying design.

4.1 PoC 1: The Closure Enclosure — Compiler-Enforced Key Destruction

Claim under test: that `SecretKeyBuffer` cannot escape the scope of `with_reconstructed_key`'s closure by any of three independent means: (a) calling a raw-bytes accessor on either key share directly, (b) returning a reference to the buffer out of the closure, or (c) reaching the buffer's raw bytes through a generic conversion trait rather than a named, unexported accessor.

Method: three separate Rust source files, each attempting exactly one of the above three escapes, compiled independently against the library crate via direct `rustc` invocation — not via an external test-harness dependency, so the verification chain has no indirection between "the compiler was asked to accept this program" and "the compiler refused." A positive control (the legitimate usage pattern, returning an owned value from the closure) is compiled alongside to confirm the harness does not reject valid usage as a byproduct of rejecting the invalid cases.

Result, verified in this document's preparation:

Escape attempt	Compiler outcome	Diagnostic
(a) Direct share accessor	Rejected	`E0599`: no method named `expose` found
(b) Return buffer reference	Rejected	lifetime may not live long enough
(c) Generic trait conversion	Rejected	`E0599`: trait bound `AsRef<[u8]>` unsatisfied

3 of 3 escape attempts rejected; 0 of 3 incorrectly accepted. The positive control (legitimate usage) compiled without modification.

```
What this proves:  the reconstructed key cannot be retained, copied, or forwarded past
its authorized closure scope by any of the three tested mechanisms, as a property of
the type system rather than of programmer discipline.

What this does not prove:  that the key is inaccessible to a physical-memory or
DMA-capable observer during legitimate execution of the closure.  The demonstration's
own runtime component confirms the reconstructed key exists as ordinary, readable
stack bytes for the closure's duration -- this is unavoidable for any software-level
mechanism and is stated here rather than obscured.
```

4.2 PoC 2: Anti-DMA Page Splitting — ELF-Level Verification

Claim under test: that `KeyShareA` and `KeyShareB`, each declared `#[repr(C, align(4096))]`, occupy two separate, non-overlapping, individually 4096-byte-aligned memory pages in the final, linked binary artifact — not merely in source-level intent.

Method: the library is compiled and linked into a real executable (a bare object file or static library does not undergo final linking and does not represent what a physical-memory reader would observe). The resulting binary is inspected at two stages using only standard Linux binutils: (1) the pre-link object file, extracted from the intermediate archive, inspected via `readelf -SW` for section-header-level confirmation that the `#[link_section]` attribute produced two genuinely distinct named sections; and (2) the final, post-link binary, inspected via `nm` and `readelf -sW` for symbol-table-level confirmation of each share's actual virtual address, size, and alignment as shipped.

Result, verified in this document's preparation, against a compiled and linked binary:

Property	SHARE_A	SHARE_B
Virtual address	0x52000	0x53000
Size	4096 bytes	4096 bytes
4096-byte aligned	Yes	Yes
Address delta	4096 bytes (exactly one page)	
Range overlap	None	

A negative control — an ordinary, non-page-aligned `u64` static included in the same binary — was confirmed, in the identical `nm`/`readelf` output, to sit at a non-page-aligned address, establishing that the two shares' alignment is a consequence of their specific type attribute rather than an artifact of the toolchain's general placement behavior.

```
What this proves:  the compiled and linked artifact's static memory layout keeps
the two key shares from being captured by a single naturally-aligned 4096-byte
physical-memory read starting anywhere within either share's own page.

What this does not prove:  that an operating system's page tables map these two
link-time pages to non-adjacent physical DRAM frames at runtime (a virtual-memory/MMU
configuration question entirely outside linker or objdump visibility, and inapplicable
in the first place on Hubris's paging-free static memory model, which is the
actual target); nor that an IOMMU or DMA-remapping unit is configured to restrict a
DMA-capable device's addressable range at any given moment.  A further, independently
discovered finding:  standard linker behavior merges same-prefixed .bss.* input
sections into one output .bss section, meaning #[link_section] does not itself
guarantee distinct output sections survive to the final binary -- the property that
does survive and that is actually load-bearing for the separation demonstrated above
is the align(4096) attribute on the type, independent of section-name merging.
```

4.3 PoC 3: Fault-Injection Immunity — 100,000-Iteration Simulation

Claim under test: that a single pseudorandomly-chosen bit flip, applied to the raw `u32` wire representation of either `GlitchResistantBool` sentinel prior to decode, can never cause `TryFrom<u32>` to silently produce the *other* defined sentinel — the exact scenario in which an adversary attempts to flip a failed verification result into a passing one via induced fault.

Method: a dependency-free pseudorandom bit-flip simulator (`XorShift32`) applied 100,000 independent single-bit flips against each of the two starting sentinels (200,000 trials total), with every corrupted value passed through the actual `TryFrom<u32>` implementation and classified into one of three outcomes: correctly rejected as `Undefined`; decoded unchanged (a defensive classification bucket, mathematically unreachable for a genuine single-bit flip against either sentinel); or a critical bypass, defined as silent decoding into the opposing defined sentinel.

Result, verified in this document's preparation:

Starting sentinel	Trials	Correctly rejected	Critical bypasses
`True` (`0x5A5A5A5A`)	100,000	100,000 (100.0000%)	0
`False` (`0xA5A5A5A5`)	100,000	100,000 (100.0000%)	0
Total	**200,000**	**200,000**	**0**

Bit-index coverage across all 32 positions was confirmed approximately uniform (observed range 3016–3258 occurrences per bit position against an expected value of 3125, consistent with unbiased pseudorandom selection across the full bit width, ruling out an accidental concentration of trials on a small subset of bit positions).

As a contrast measurement, the identical fault model (100,000 single-bit flips) was applied to a naive one-bit-apart encoding (`False = 0`, `True = 1`, a pattern common in unexamined

boolean-result code) under the interpretation rule "any nonzero value reads as true," a standard and unremarkable pattern in ordinary systems code:

Encoding	Trials	Silent bypass rate
`GlitchResistantBool` (Hamming distance 32)	100,000	0.0%
Naive 0/1 encoding	100,000	100.0%

What this proves: across 200,000 independent single-bit-flip trials against the actual TryFrom<u32> implementation, zero trials resulted in a silent security-check bypass, and the mechanism's advantage over an unexamined naive encoding is not asserted but measured, under an identical fault model, at 100 percentage points.

What this does not prove: protection against a fault landing on an already-decoded value sitting in a register or memory location after a successful decode has already occurred -- a fault-coverage question the specification identifies as a hardware concern (ECC, redundant read-back), outside the reach of any type-level software mechanism, and outside what this simulation measures or claims to measure.

4.4 Summary Position

The three results above are structural facts about compiler and linker behavior against real, executed artifacts, not projections from the specification's prose. Each carries an explicit boundary rather than an implied one. The security boundary these three PoCs collectively establish — key-destruction enforcement, physical page separation, and decode-time fault immunity — is built and independently verified. The runtime business logic that would connect these three boundaries into an operating cryptographic boundary controller is not yet written, and this document does not claim otherwise at any point.

5 Author's Note and Open Call for Collaboration

I am not, by background or by trade, a traditional software developer. My role in producing the Ghost-Hunter v1.0.18 specification and the three proofs of concept described in Section 6 has been that of a systems architect: modeling an adversarial threat environment in precise, falsifiable detail; identifying which specific engineering mechanisms (type-system constraints, linker attributes, exhaustive error typing) map onto which specific threats in that model; and using AI-assisted engineering tools to translate that architectural reasoning into working Rust code, compiled and verified against a real toolchain rather than left as an unimplemented diagram. The specification and the PoCs in this document are the direct output of that process, and I am stating the process plainly rather than presenting the artifact without context.

This document has, at every section, drawn a clear line between what is specified, what is proven, and what remains to be built. That line is not a weakness to be managed in conversation — it is the actual, current state of the project, and I would rather have it read accurately by the right person than rounded up and discovered later by the wrong one.

What exists today is a threat model I believe is under-addressed by current commercial and defense-adjacent cryptographic infrastructure, a specification that responds to it mechanism-by-mechanism rather than with a single monolithic claim, and three independently reproducible proofs that the specification's core structural claims survive contact with a real compiler, a real linker, and a real 200,000-iteration fault simulation. What does not yet exist is the runtime: the HMAC computation logic, the Hubris IPC server implementations, the hardware bring-up against the reference N4500 platform, and the integration work connecting the three proven boundaries into a single operating system.

I am seeking to work with:

- **Deep-tech venture capital investors** evaluating pre-implementation, specification-stage security architecture as an investment category, with the explicit understanding that this is a pre-runtime engineering effort requiring committed technical execution to reach a deployable state.
- **Rust systems and OS-development engineers**, particularly those with `no_std`, embedded, or microkernel experience, capable of taking on a technical leadership or CTO-track role in building the runtime this specification currently lacks.
- **Defense-technology firms** for whom the threat model described in Section 1 — hostile silicon, physical fault injection, Ring −3 adversaries — is an existing, unmet procurement requirement rather than a hypothetical scenario.
- **Intelligence-community research labs** with the technical depth to evaluate this specification's claims directly against their own threat models and red-team methodology, and the interest in doing so collaboratively rather than adversarially.

If the threat model described in this document matches a problem you are already trying to solve, or if you have the technical capability to help build the runtime this specification currently lacks, I would like to hear from you directly.

Taha Kouiyasse
Systems Architect
tahakouiyasse@protonmail.com